Software Systems

Session 6

Behavioral Modeling

Y. Raghu Reddy
Software Engineering Research Center
IIIT Hyderabad, India

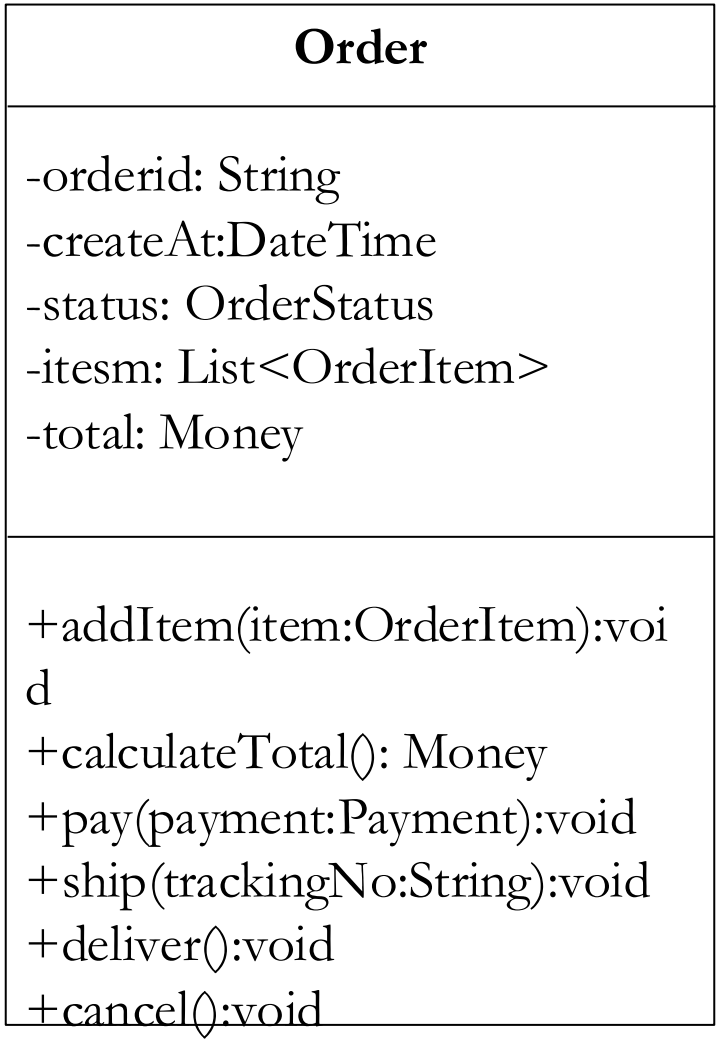
INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY
HYDERABAD

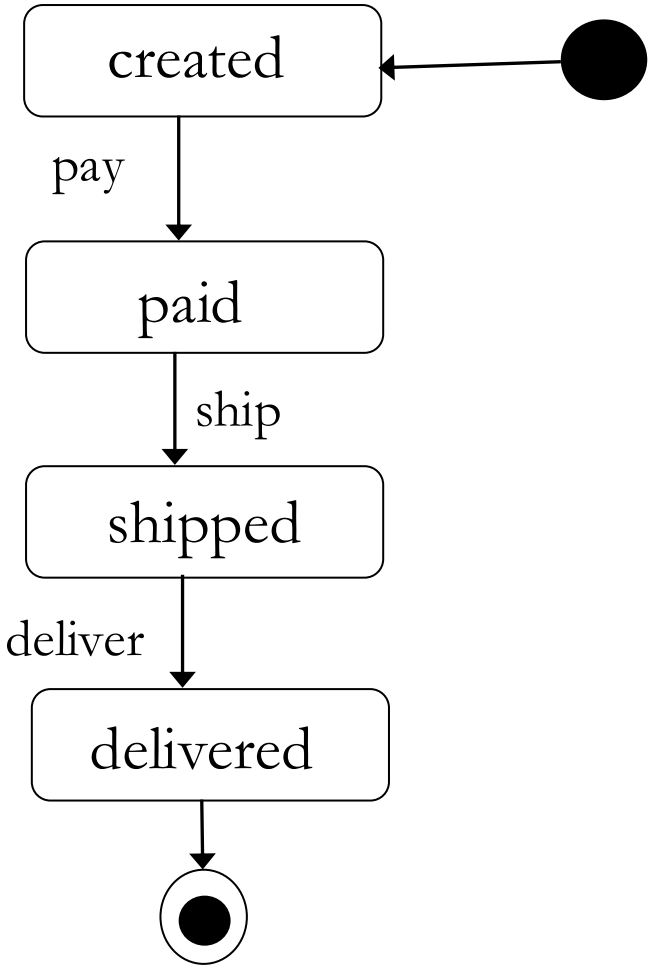
From Requirements to Behavioral Modeling: Understanding How Systems Act Over Time

SDLC → Planning → Requirements → Modeling → UML (Class + Sequence) → **Behavioral Modeling (today)**

"Let's begin by asking: why do we even need behavioral models when we already have structure and interactions captured?"

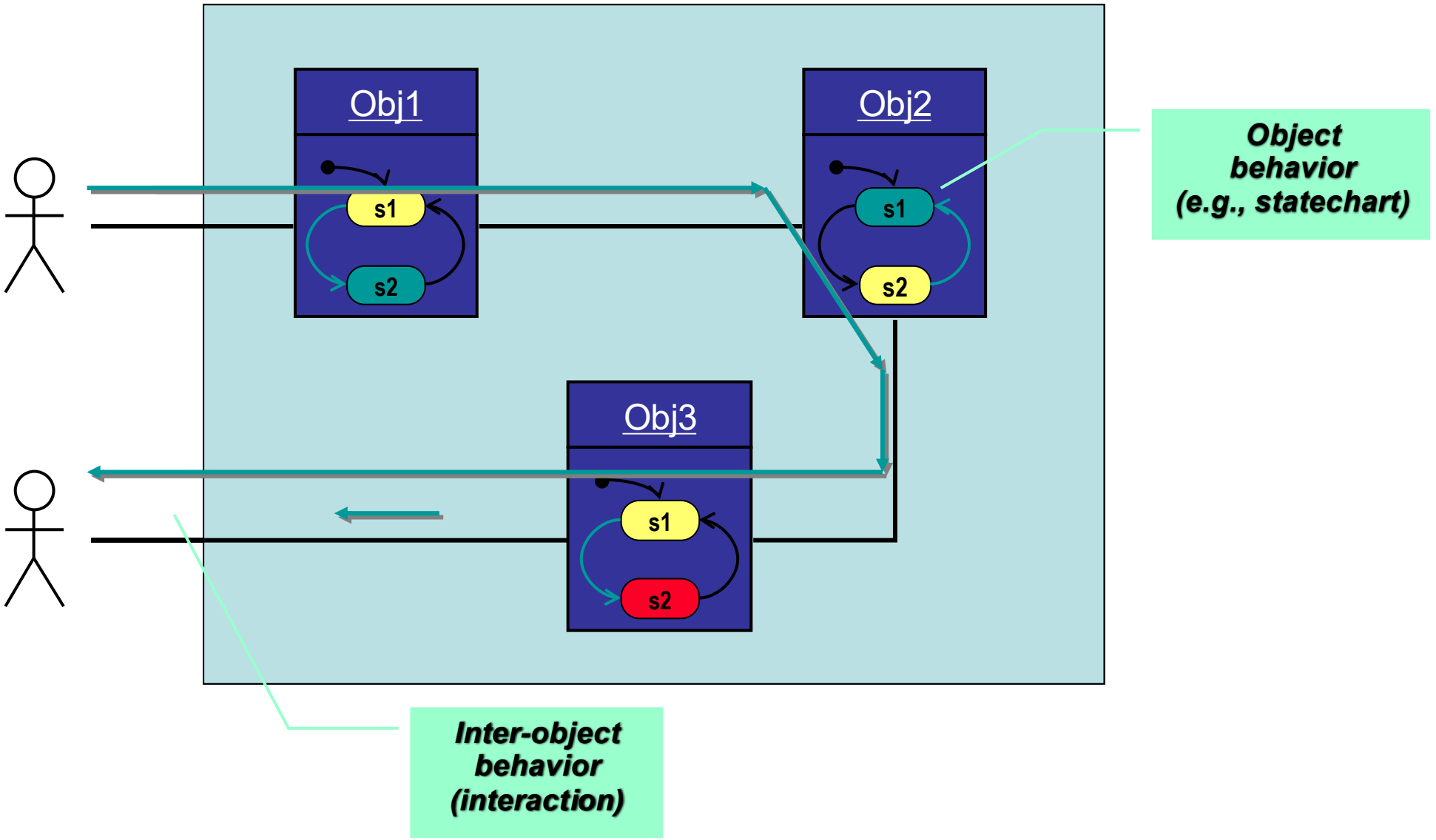
Why structural & interaction models are not enough?



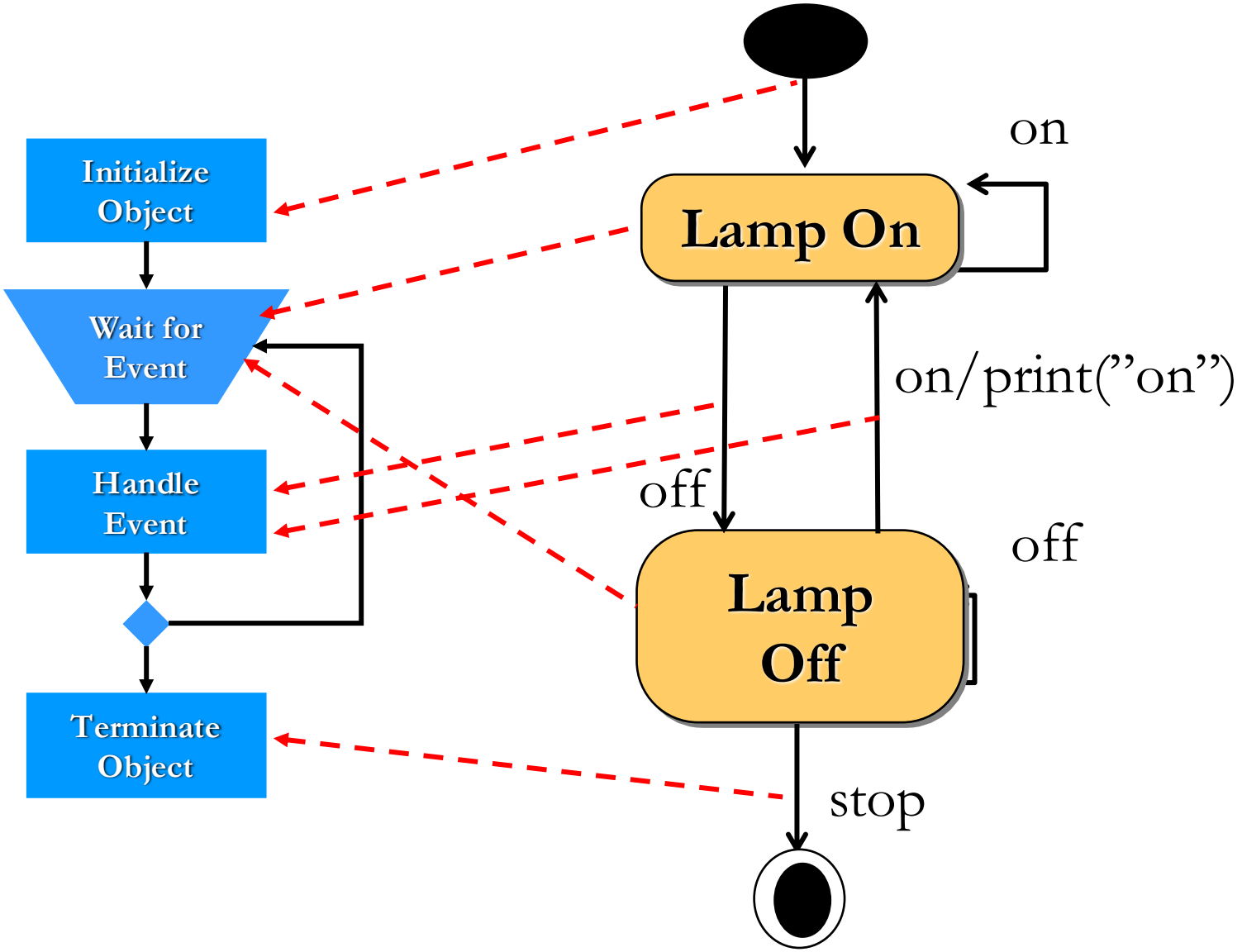


Specifying interactions/behavior using UML

- In UML, all behavior results from the actions of (active) objects



Object Behavior and State diagram



Reflecting object lifecycle in a state diagram

State diagrams

- A *state diagram* specifies the life histories of objects in terms of the sequences of operations that can occur in response to external stimuli.
  - For example, a state diagram can describe how an object responds to a request to invoke one of its methods.

- A state diagram describes behavior in terms of sequences of *states* that an object can go through in response to *events*.

Key Concepts

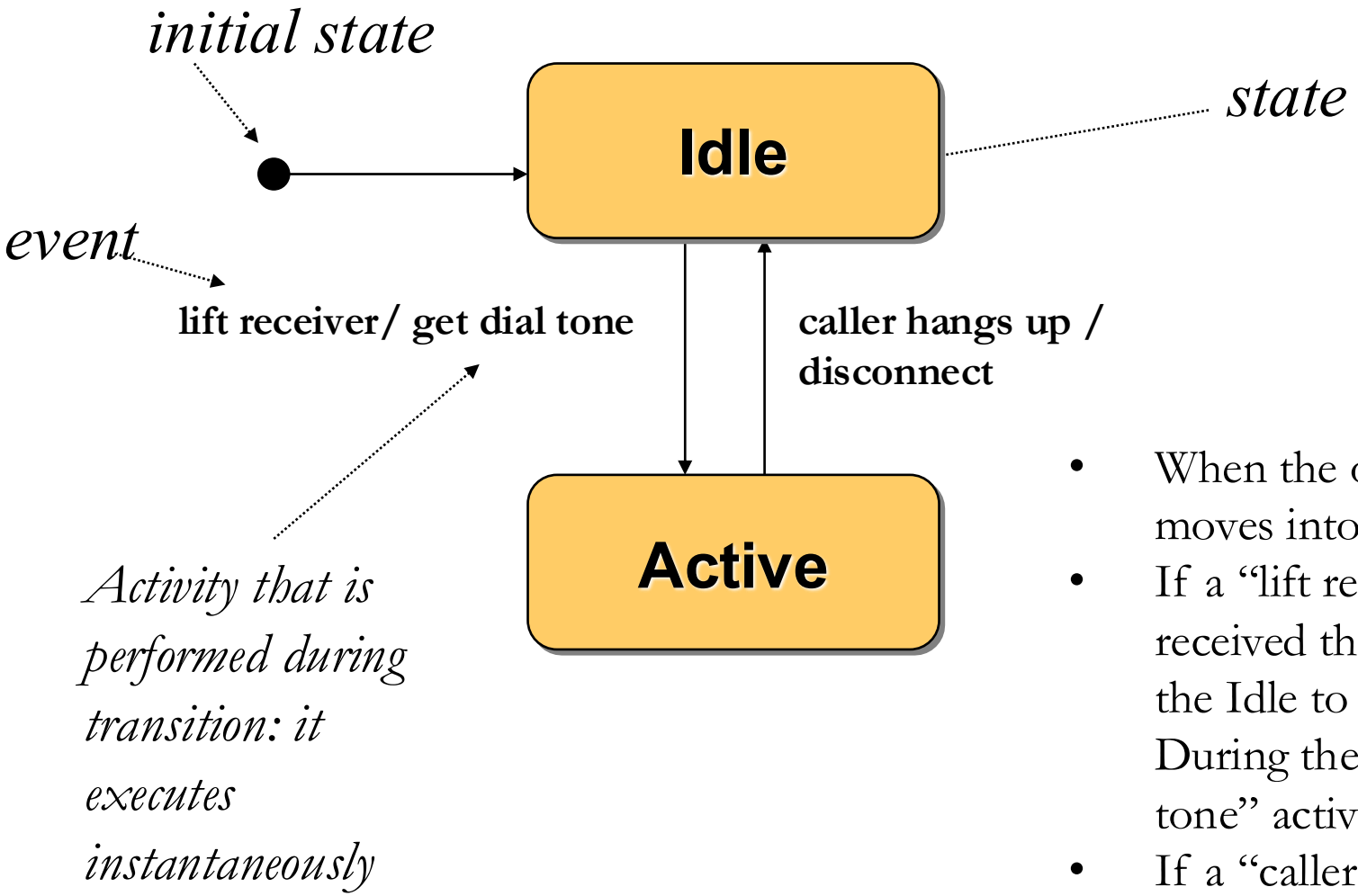
- An *event* is a significant or noteworthy occurrence at a point in time.
  - Examples of events: sending a request to invoke a method, termination of an activity.
  - An event occurs instantaneously in the time scale of an application.

- A *state* is a condition of an object during its lifetime.
  - For example, a student is in the registered state after completing course registration.
  - A state is an abstraction of an object's attribute values and links
    - For example, a bank account is in the Overdraft state when the value of its balance attribute is less than 0.

Key Concepts - 2

- A *transition* occurs when an event causes an object to change from its current (source) state to a target state.
  - For example, if a student is in the registered state and then drops out of the program then the student is in the "not registered" state.
  - The source and target states can be the same.
  - A transition is said to fire when the change from source to target state occurs.
- A *guard condition* on a transition is a boolean expression that must be true for a transition to fire
- An *activity* is a behavior that is executed in response to an event.

Simple Example: Telephone Object



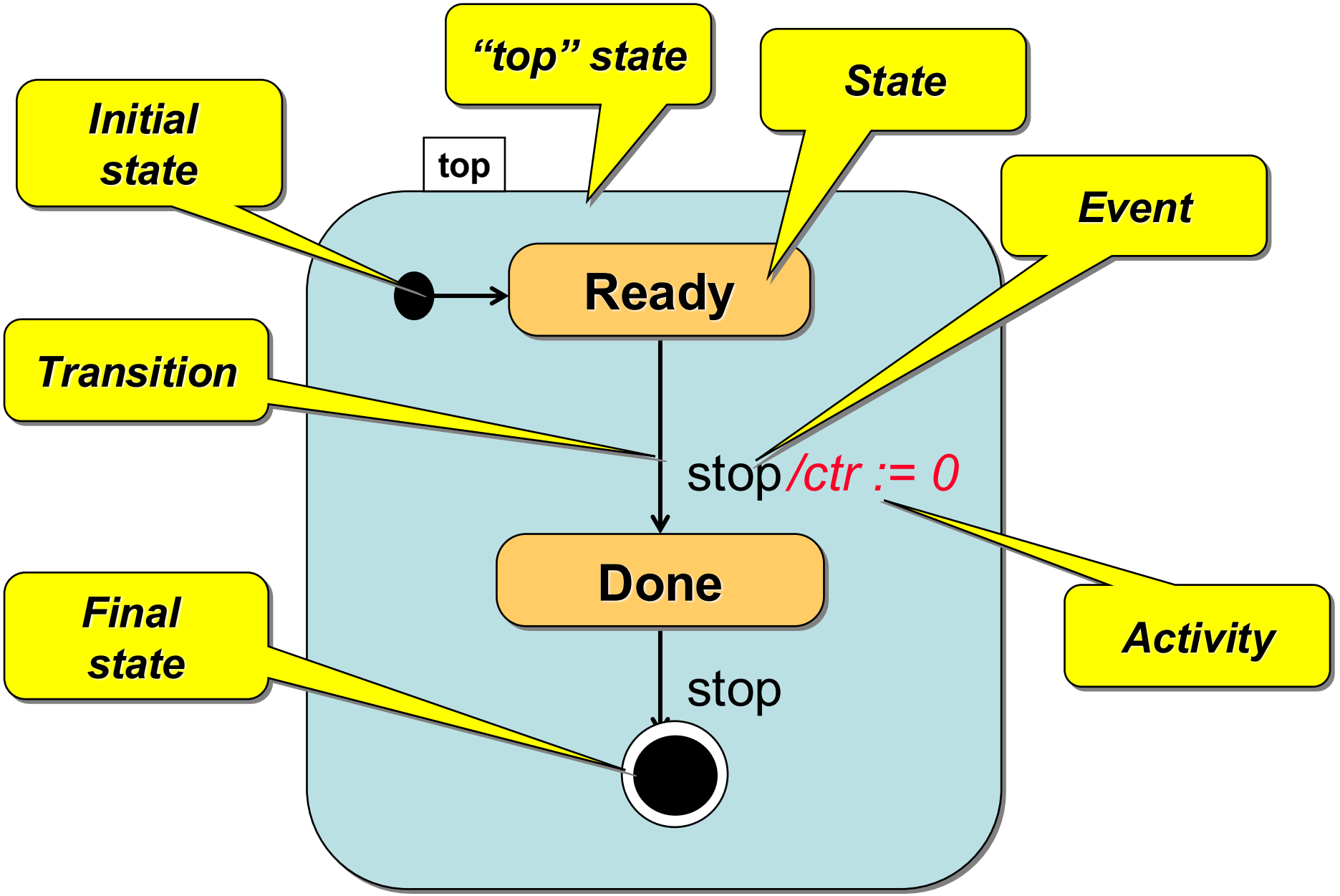
- When the object is created, it moves into the Idle state.
- If a "lift receiver" event is received the object moves from the Idle to the Active state. During the transition the "get dial tone" activity is executed.
- If a "caller hangs up" event occurs when in Active state, the object moves to the Idle state and the "disconnect" activity is performed during the transition.

Simple Transitions

- A transition is a relationship between two object states indicating that an object will move from one state to the other and perform certain actions when a specified event occurs.

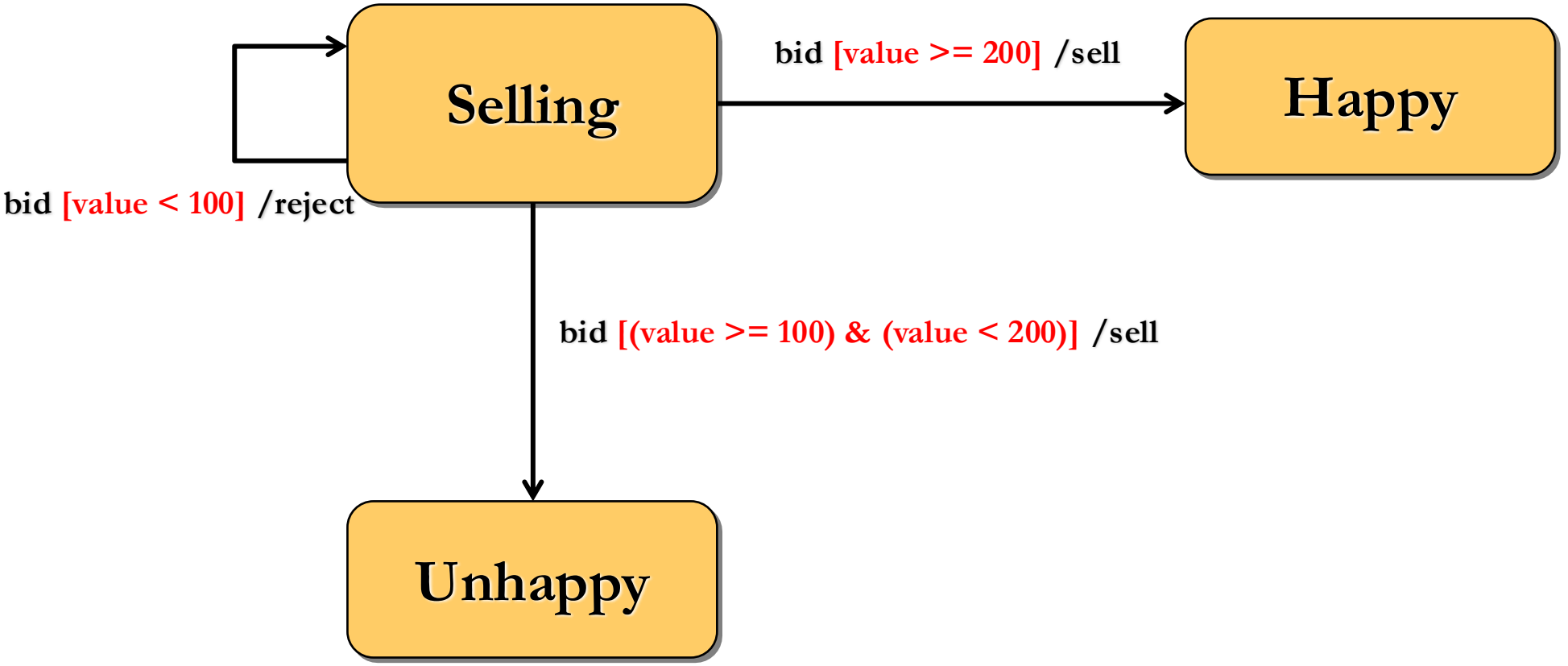
Event(args)[condition]/activity^destination.message(params)

Basic UML State Diagram



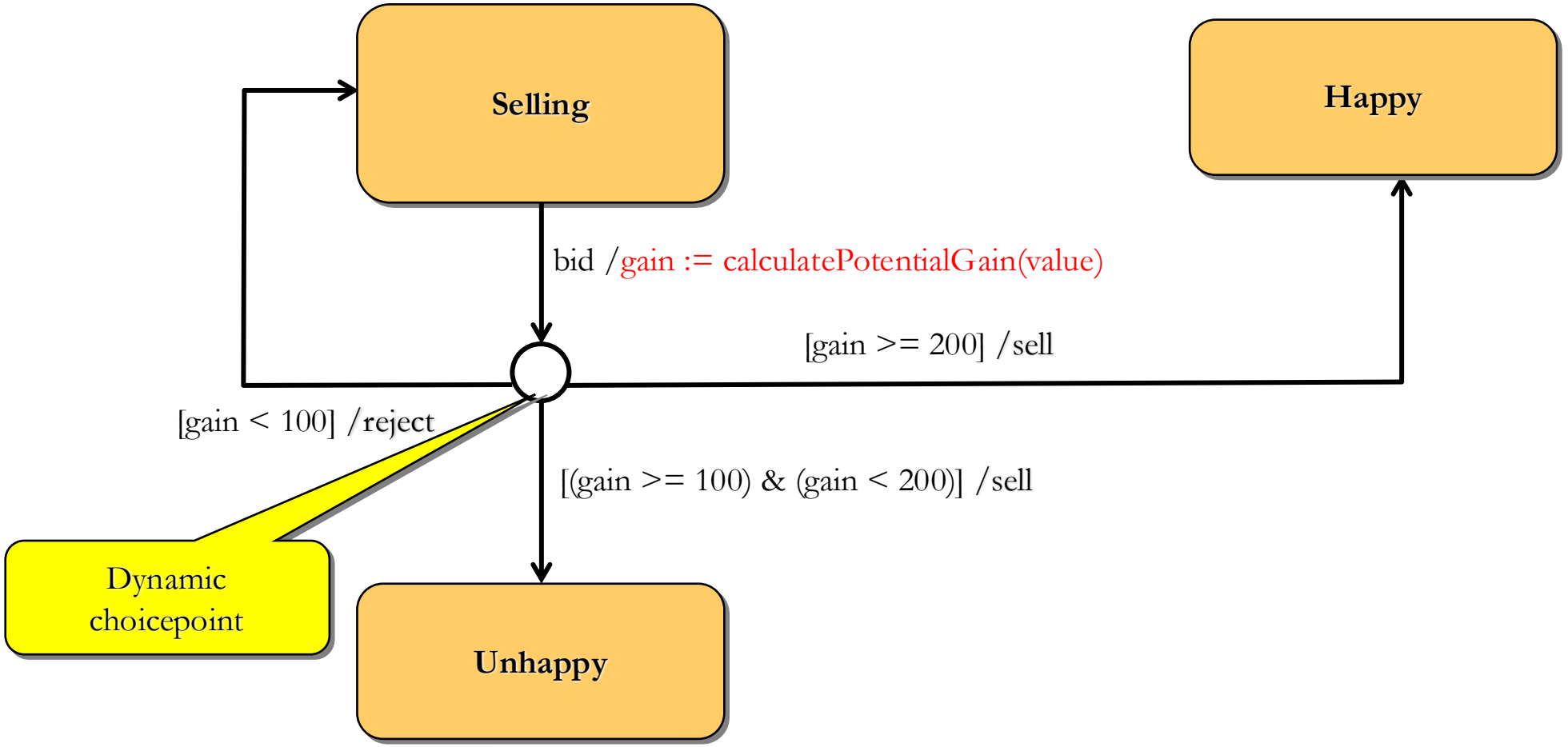
Guards

- Conditional execution of transitions
  - guards (Boolean predicates) must be side-effect free

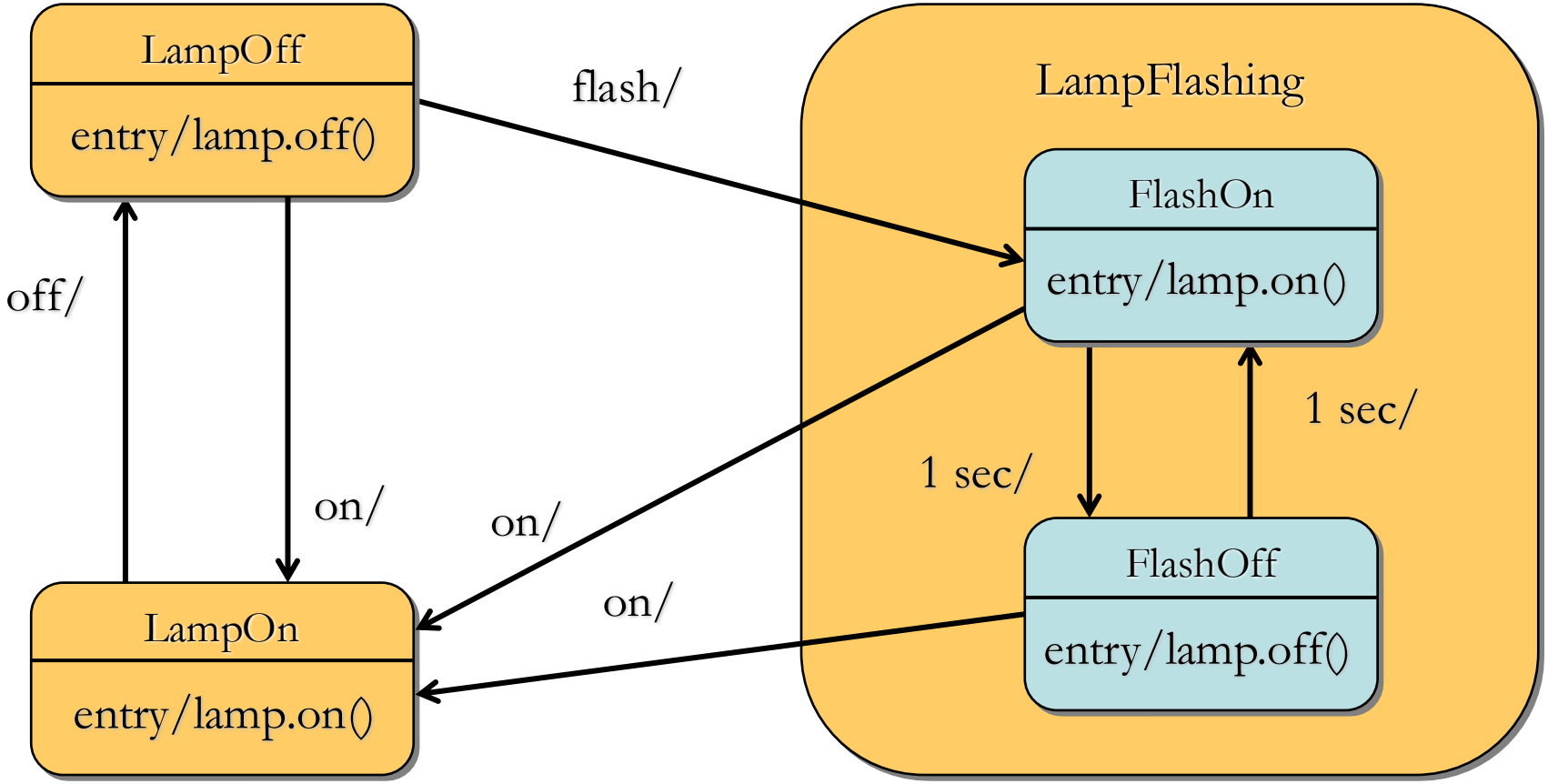


Dynamic Conditional Branching

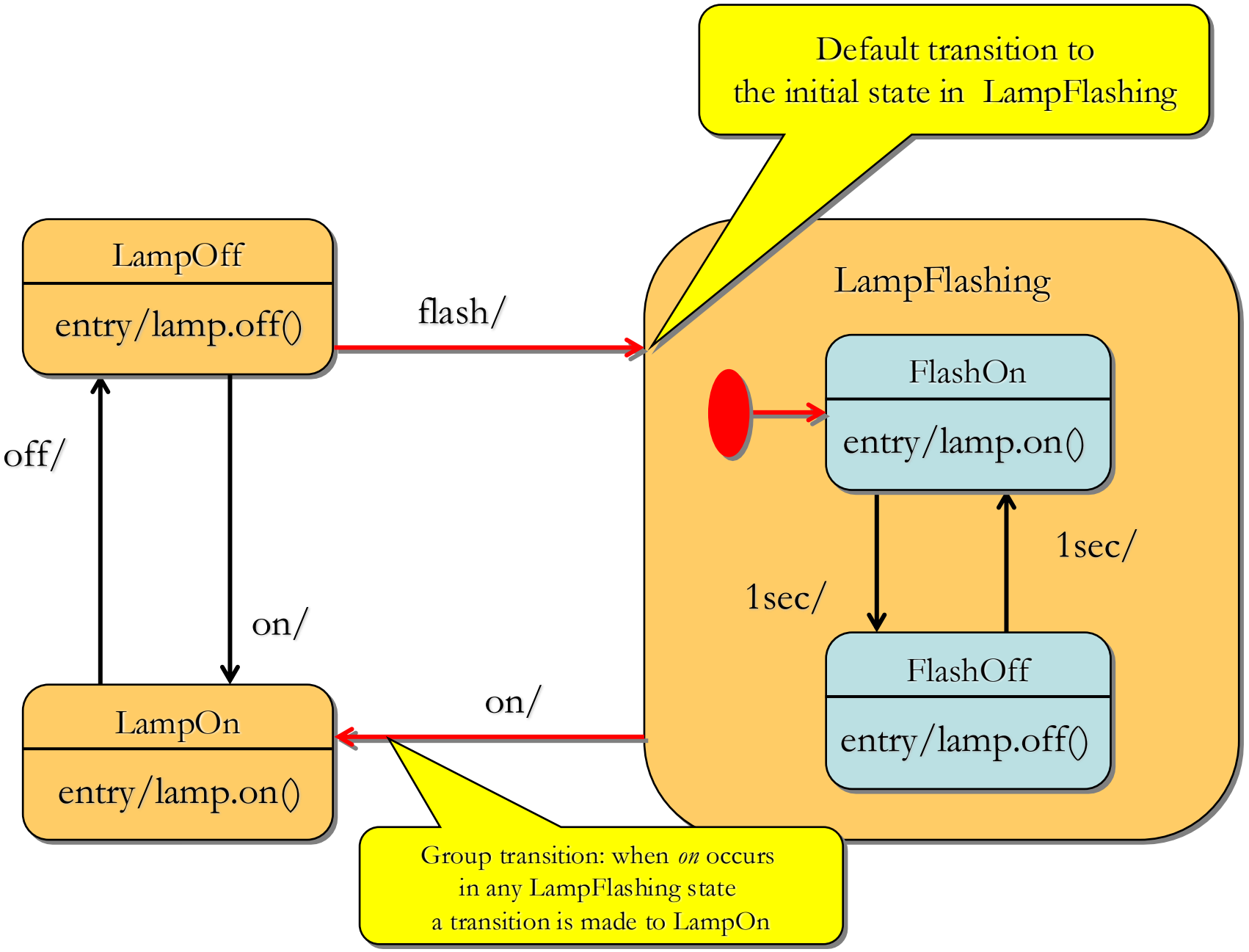
Choice pseudostate: guards are evaluated at the instant when the decision point is reached



Hierarchical State Machines

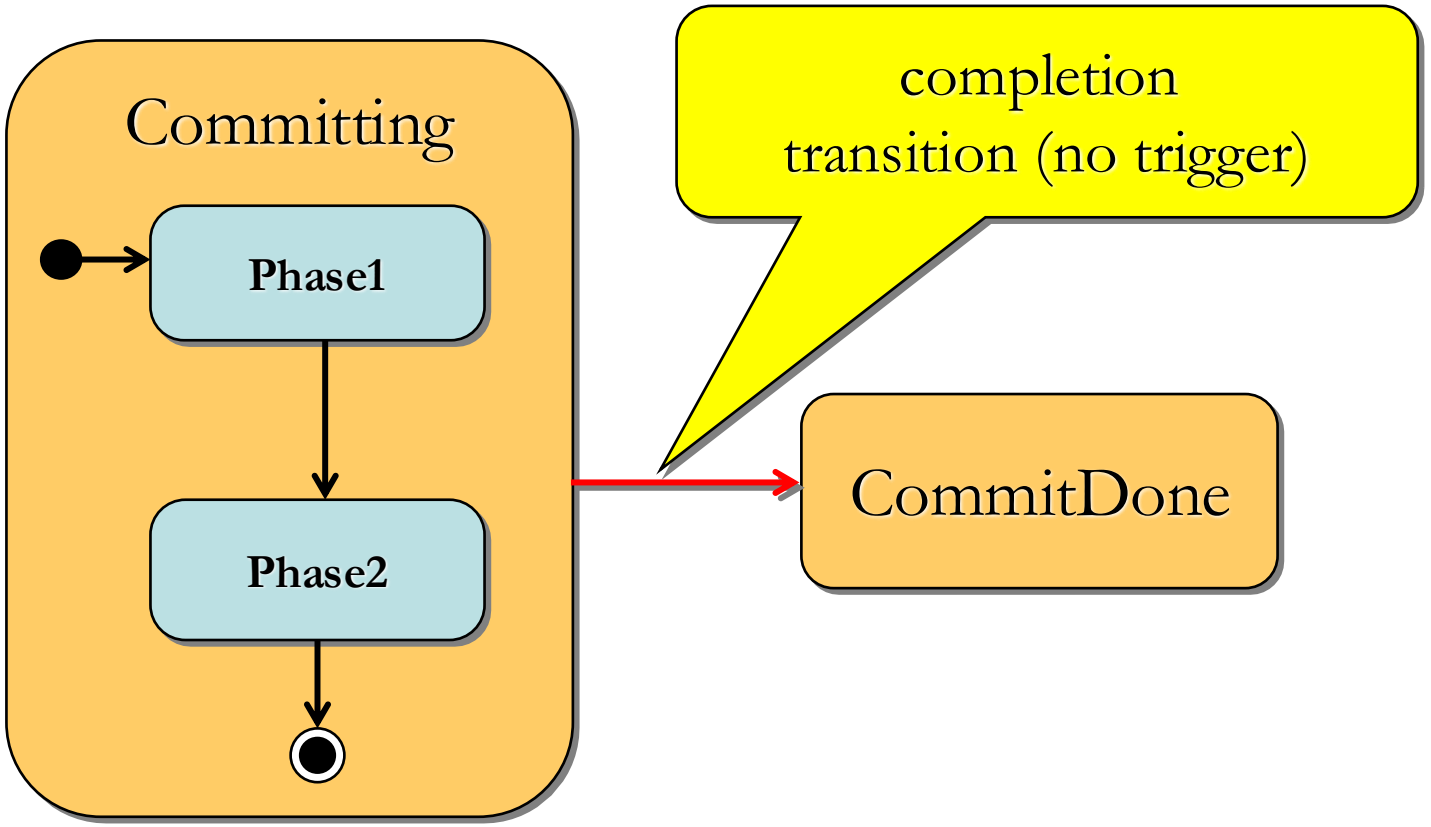


Group Transitions



Completion Transitions

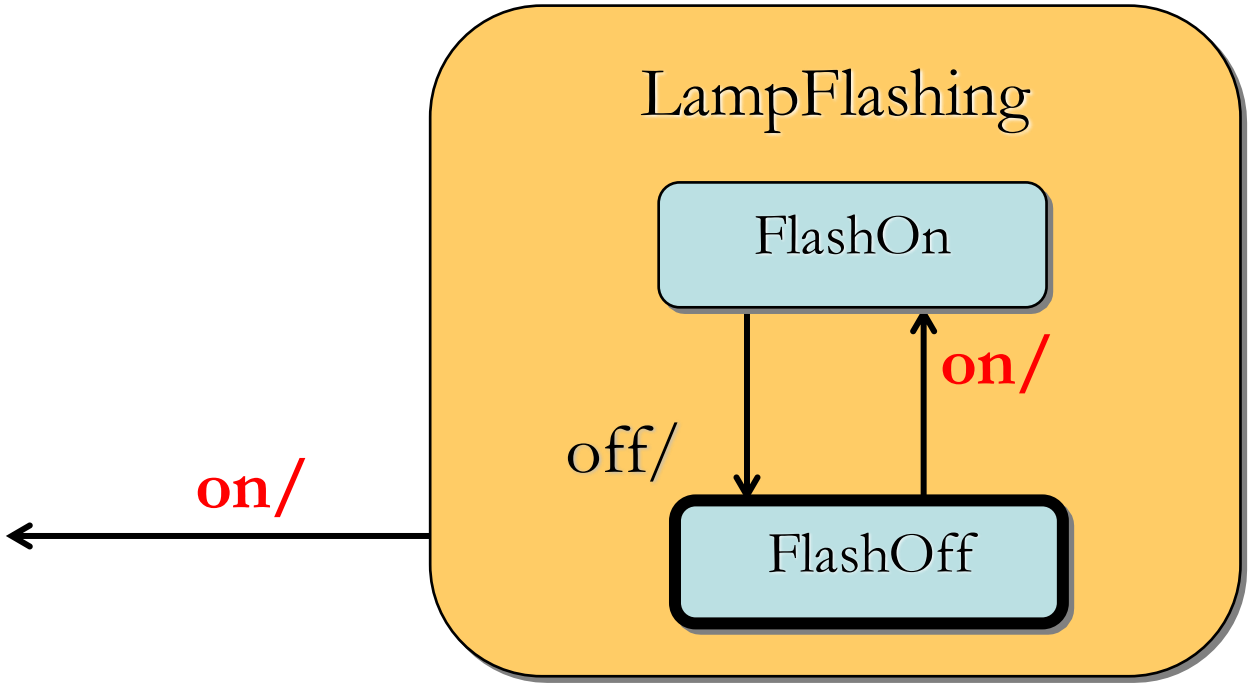
- Triggered by a completion event
  - generated automatically when an immediately nested state machine terminates



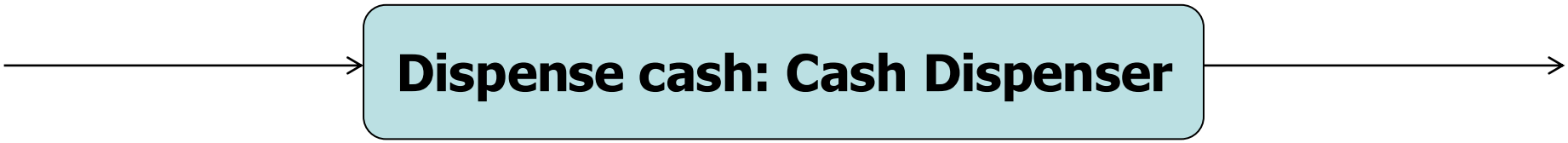
Triggering Rules

Two or more transitions may have the same event trigger

- innermost transition takes precedence
- event is discarded whether or not it triggers a transition



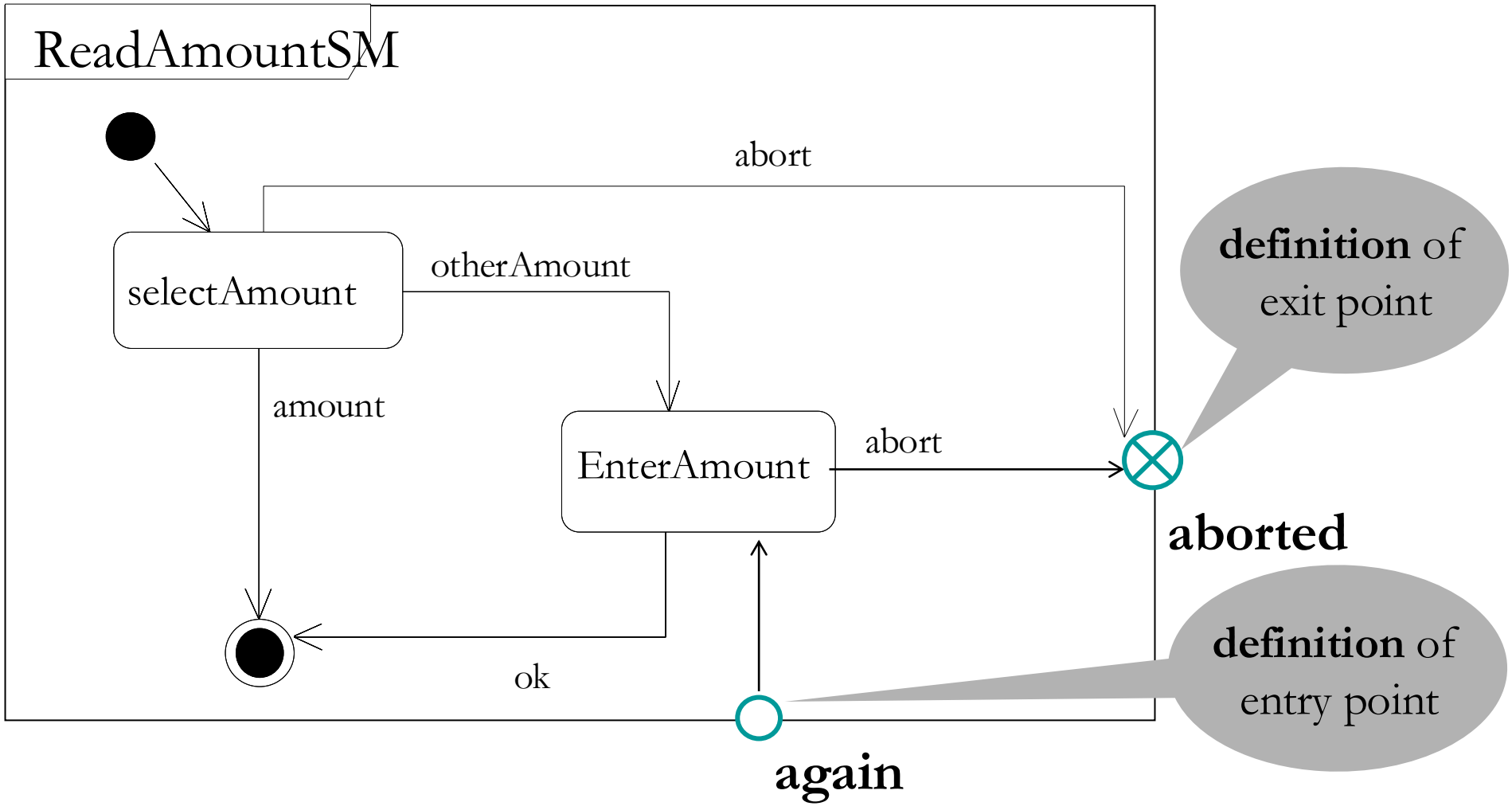
State Syntax - Submachines



- *State name:submachine*
  - machine is the name of a (nested) state machine that has an initial and final state.
  - Execution of the nested machine begins in the initial state.
  - When final state of the nested machine is reached then the exit action of the parent state is executed (if it exists).

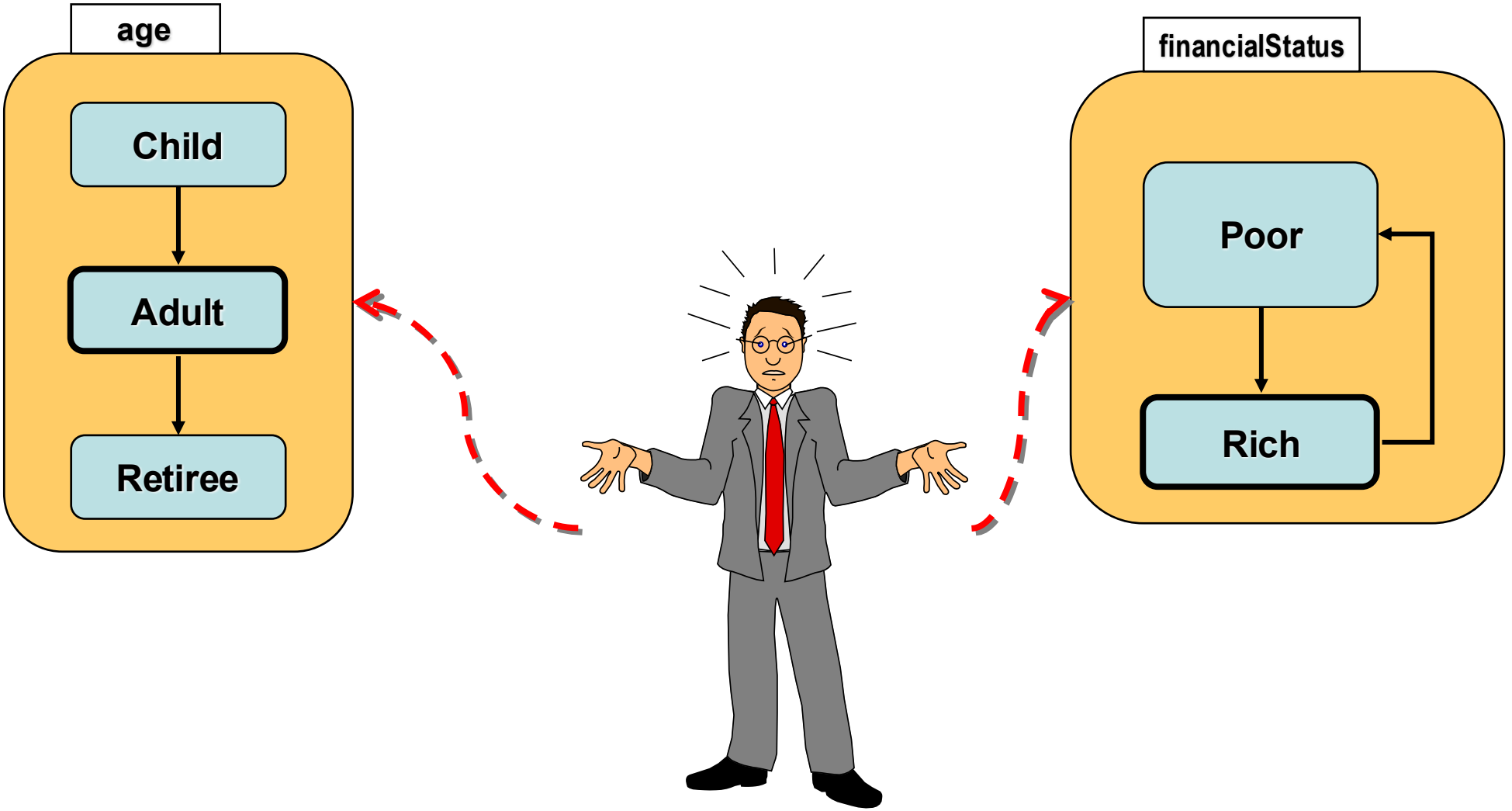
UML 2.0: Entry/Exit Points

Encapsulation of submachines



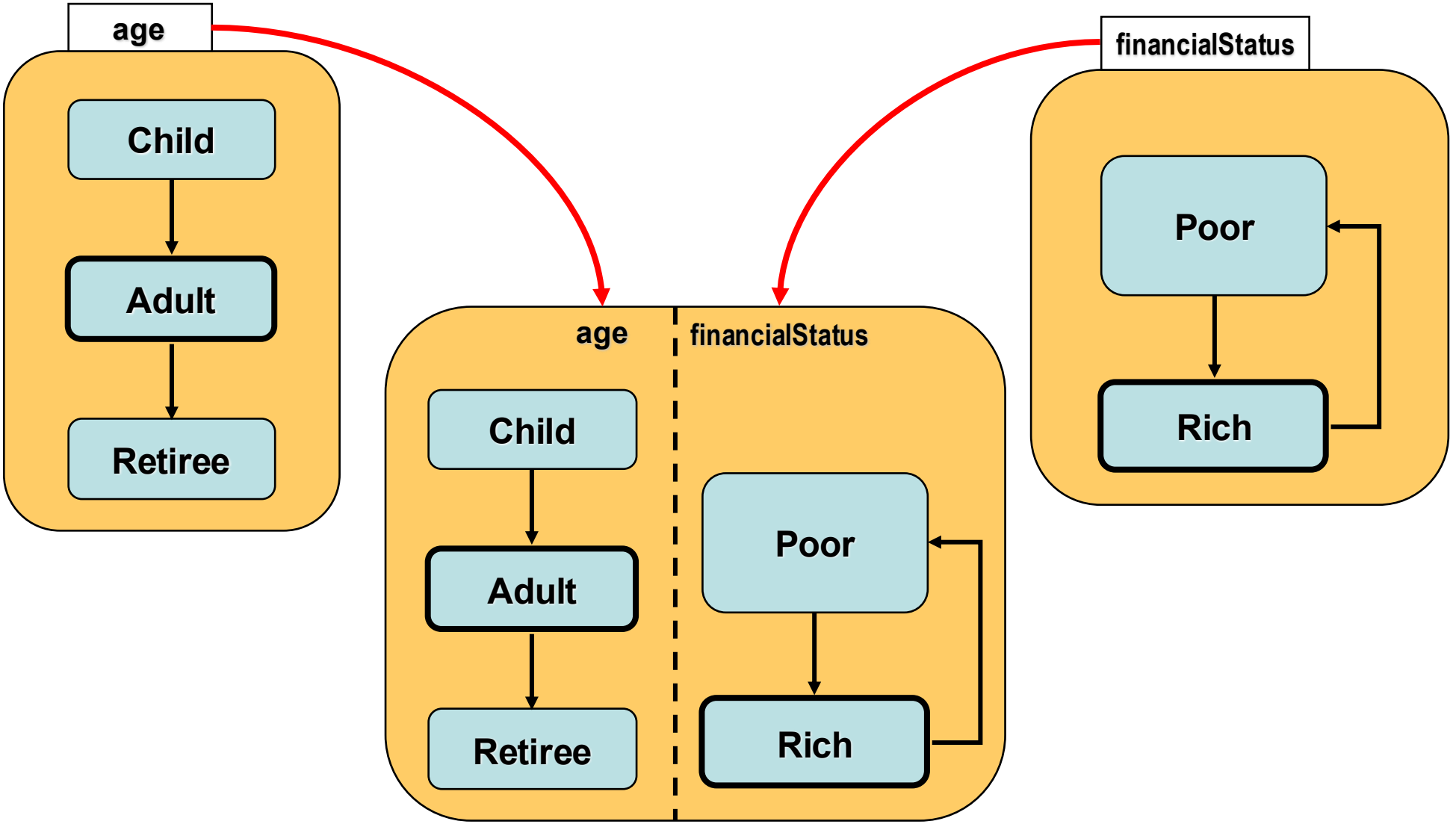
Orthogonality

Multiple simultaneous perspectives on the same entity



Orthogonal Regions

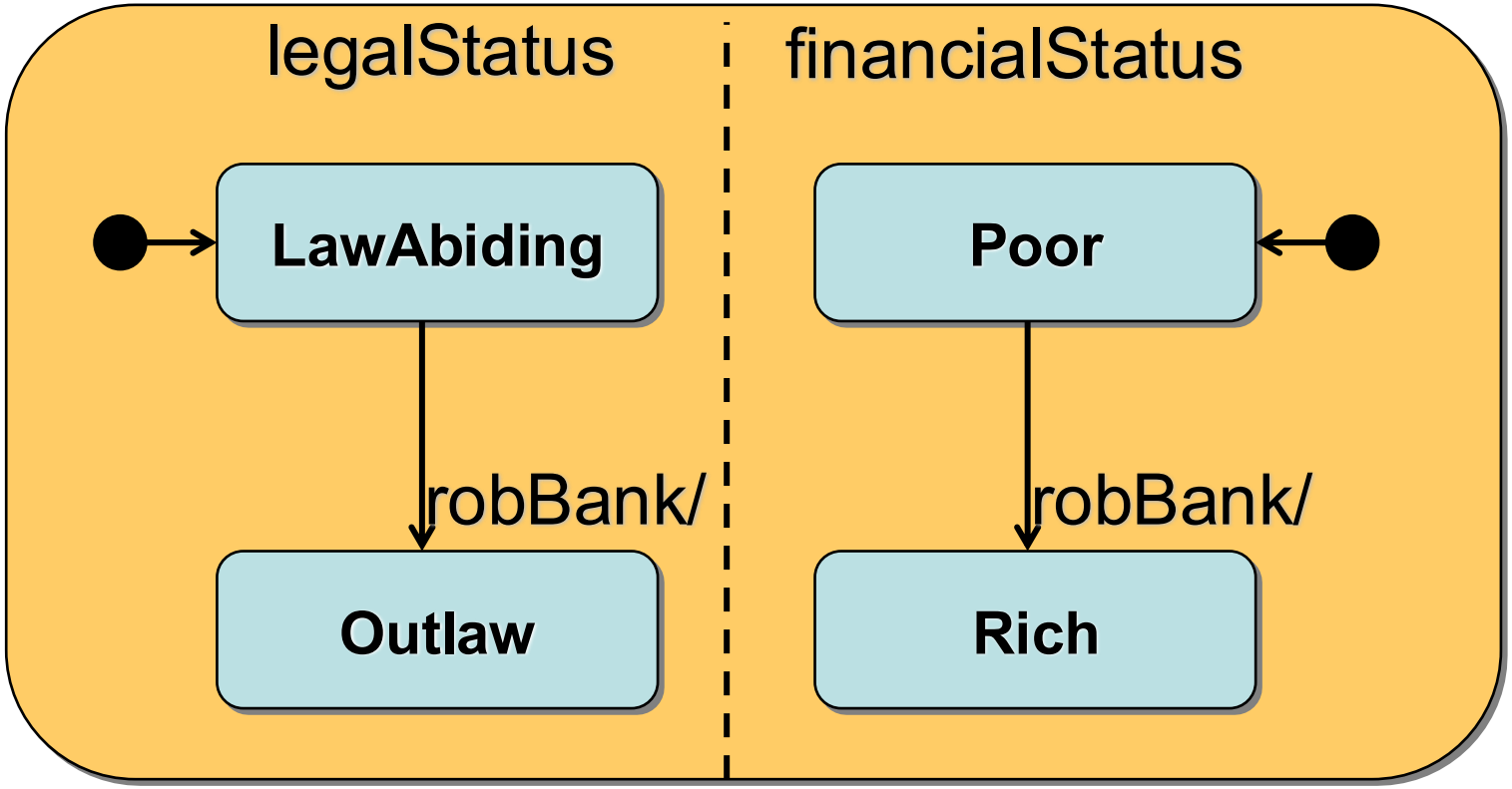
Combine multiple simultaneous descriptions



Orthogonal Regions - Semantics

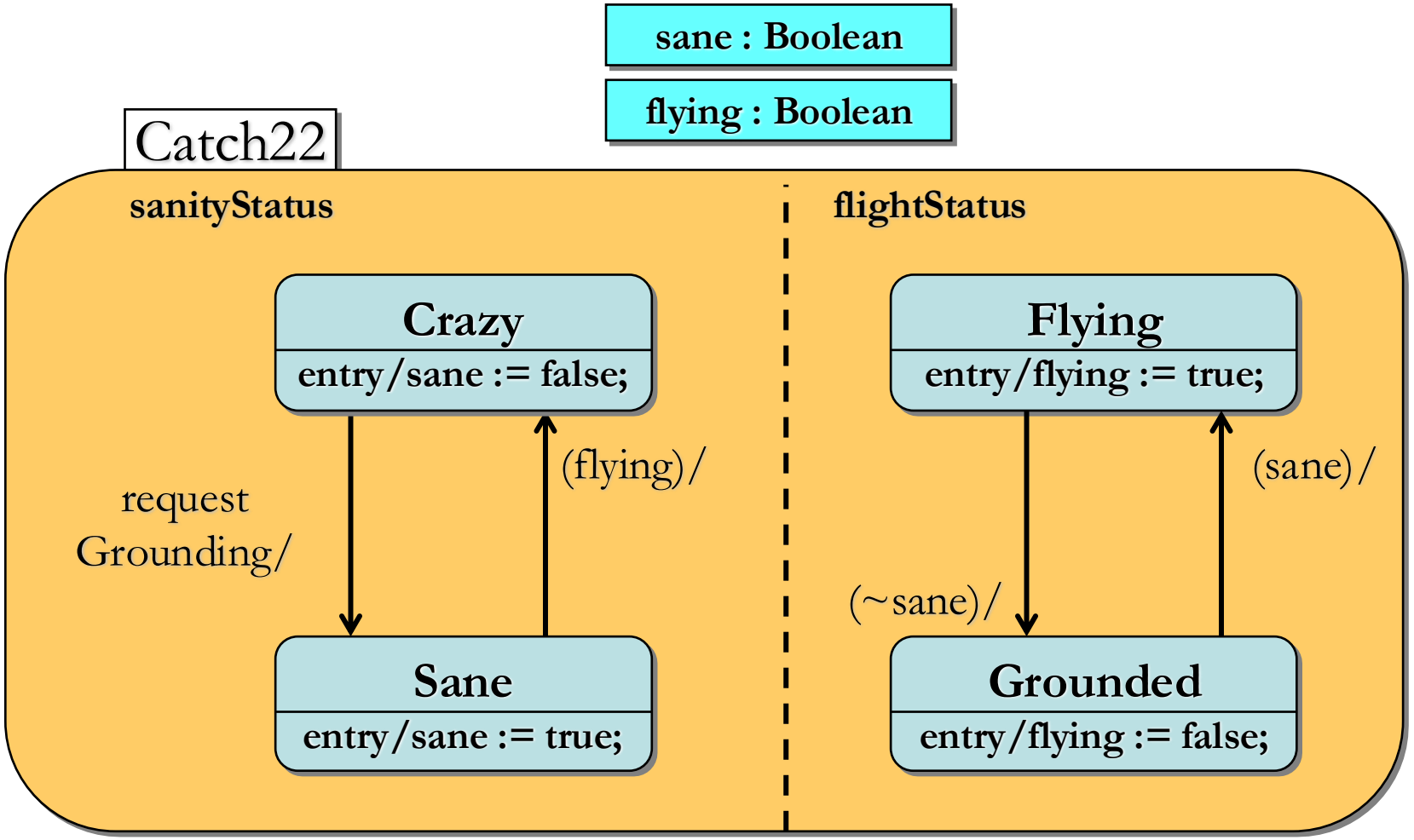
All mutually orthogonal regions detect the same events and respond to them "simultaneously"

- usually reduces to interleaving of some kind



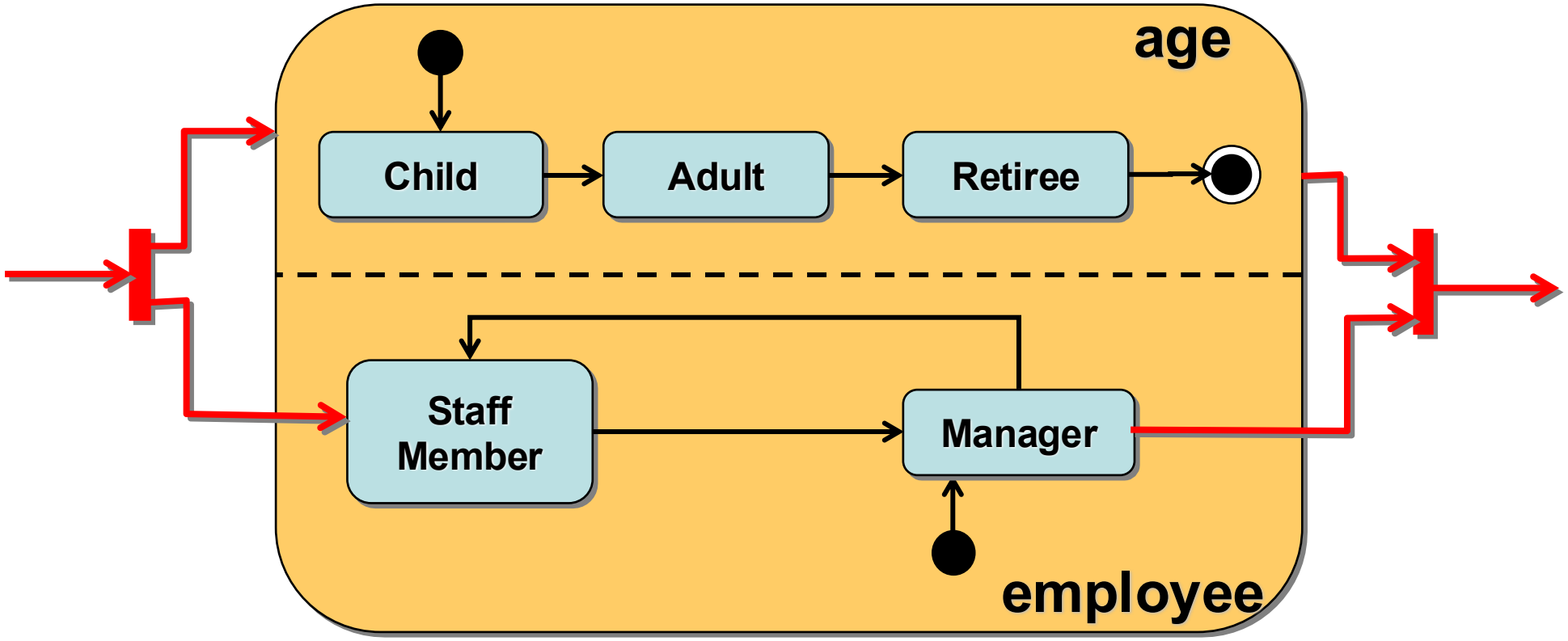
Interactions Between Regions

Typically through shared variables or awareness of other regions' state changes

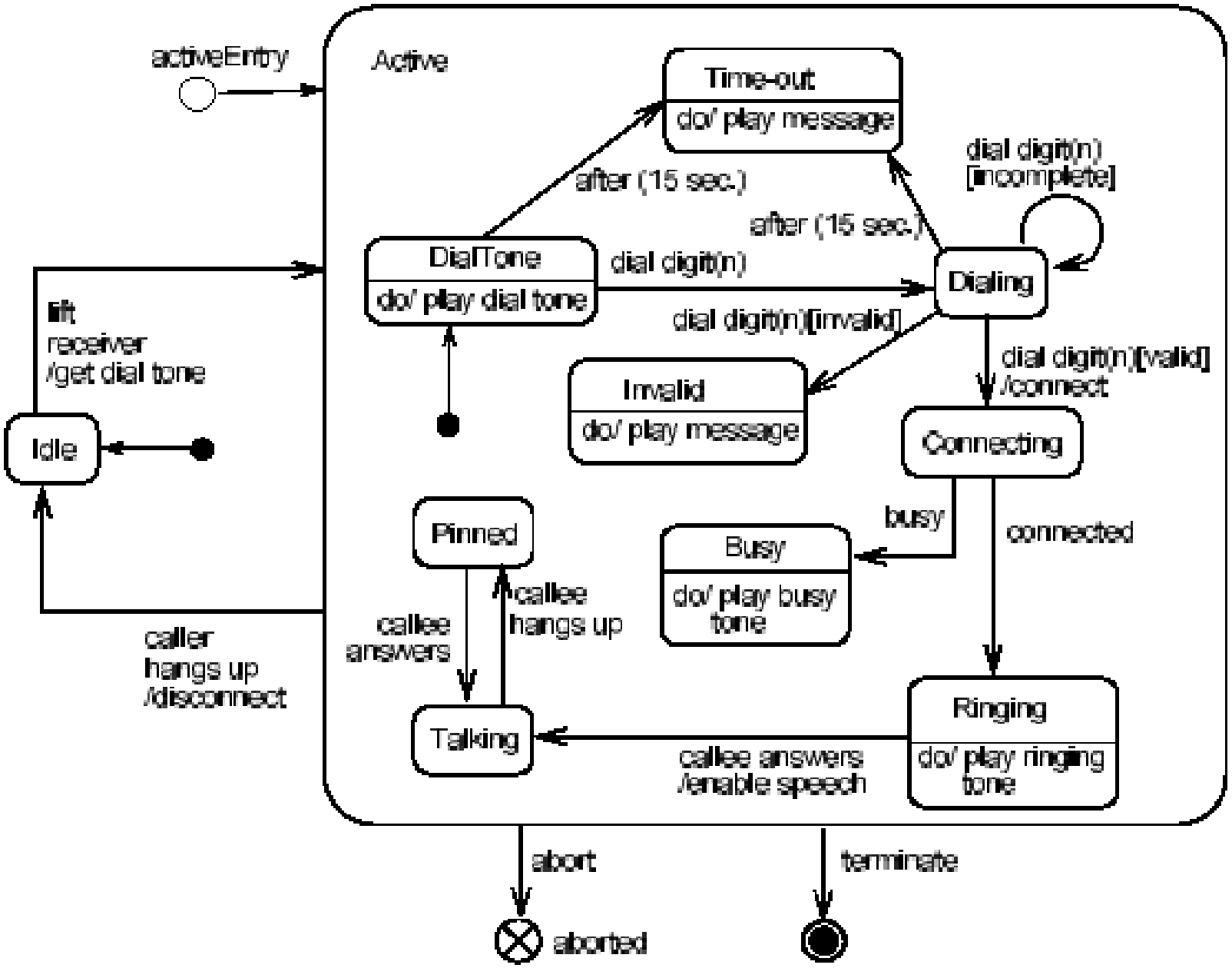


Transition Forks and Joins

For transitions into/out of orthogonal regions

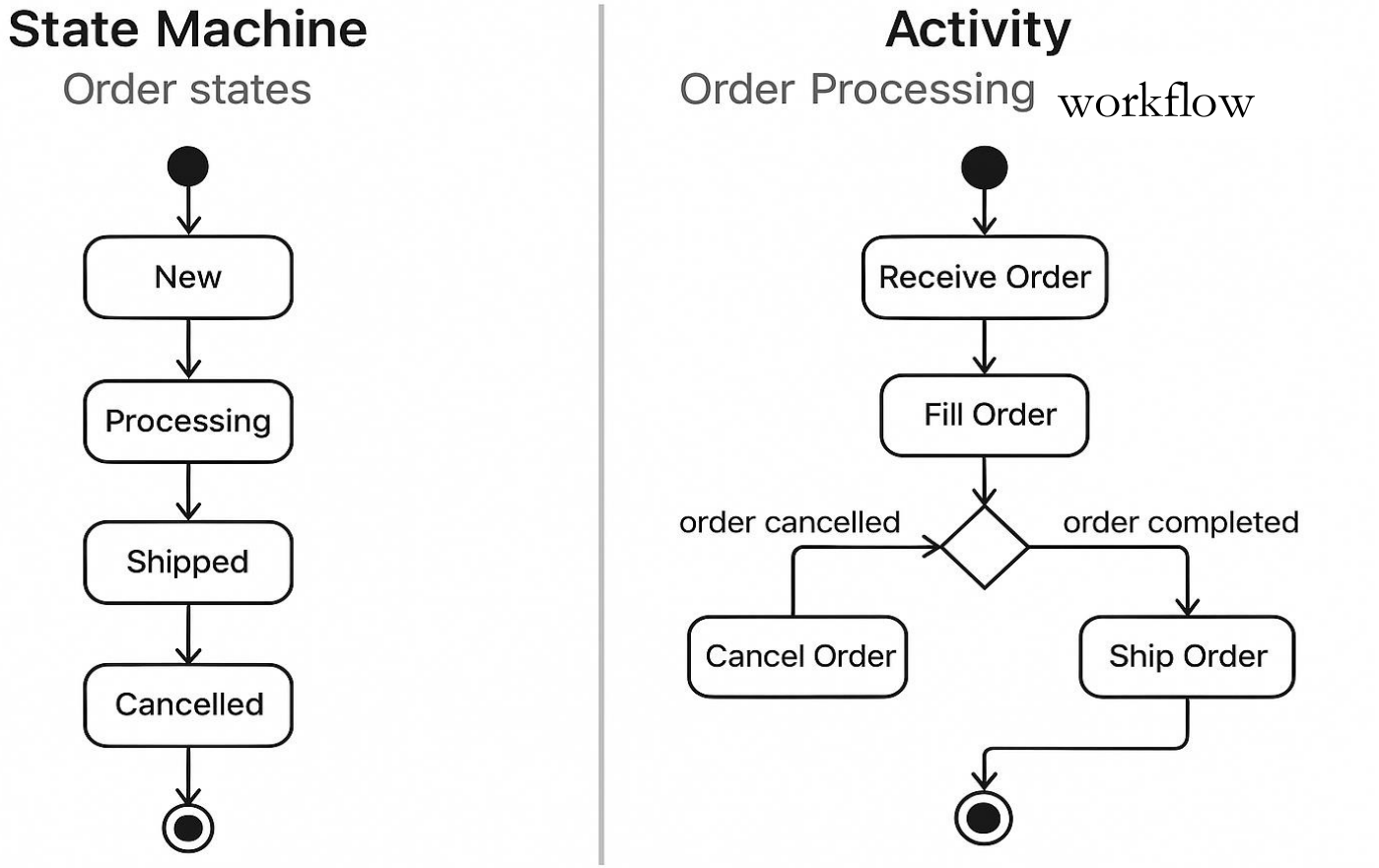


Telephone state machine - Example



Thank you !

From Object Behavior to System workflows



- State machines tell us what one object does. But in real systems, several entities cooperate in a process — ordering, payment, notification.
- Activity diagrams help us visualize such workflows

Activity Diagrams

- UML Activity diagrams can be used to show sequential and parallel activities in a process. They are useful for modeling business processes, workflows, data flows, and complex algorithms
  - The diagram can show both *control* flow and *data* flow

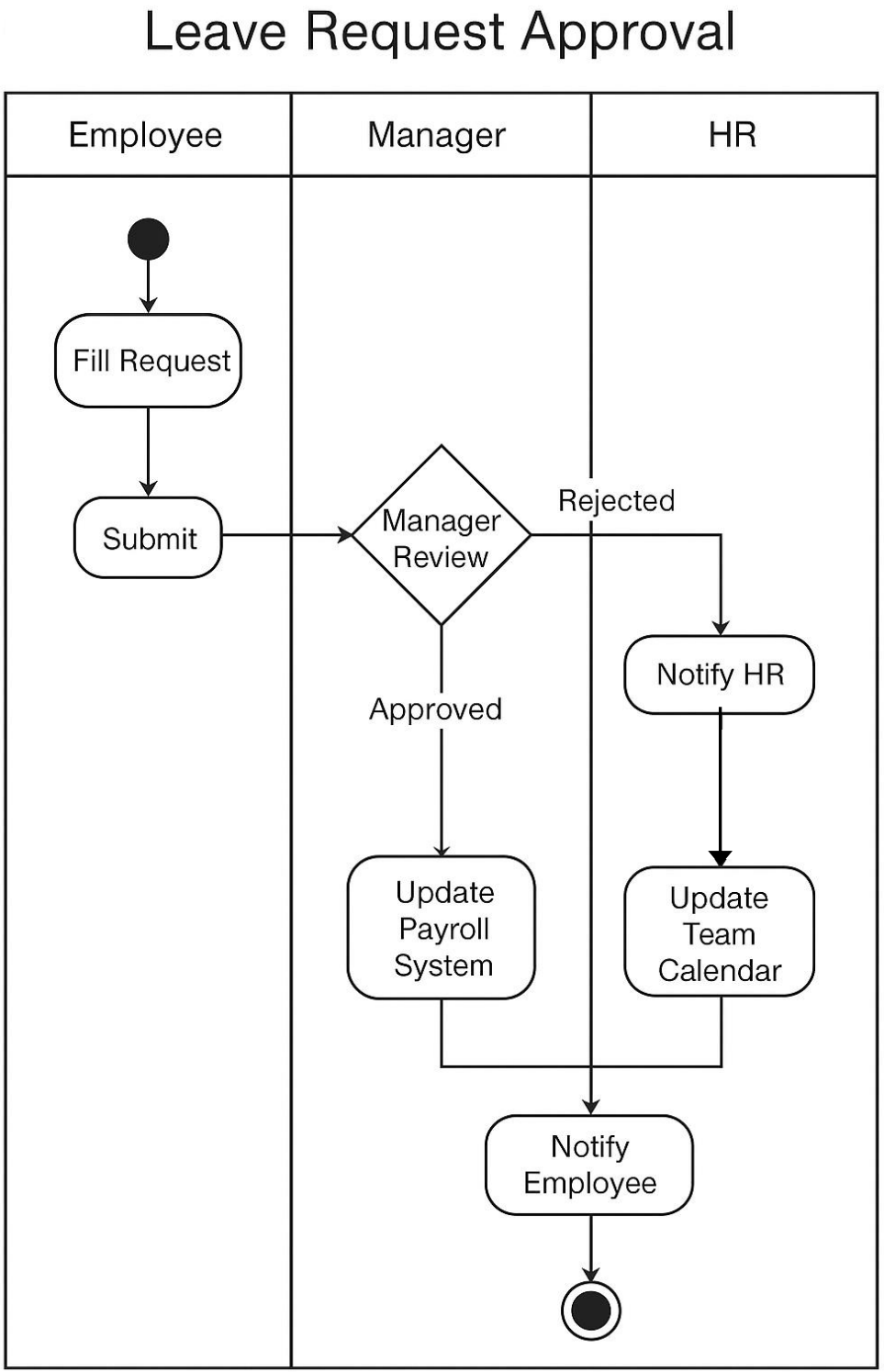
Core Elements of an Activity Diagram

Symbol	Meaning	Example
●	Initial Node	Workflow starts when request is received
◇	Decision / Merge	Payment successful?
⊤	Fork / Join	Parallel branches (e.g., send email & update database)
Rounded Rectangle	Action / Activity	"Validate Login", "Process Payment"
Arrow	Control Flow	Order of execution

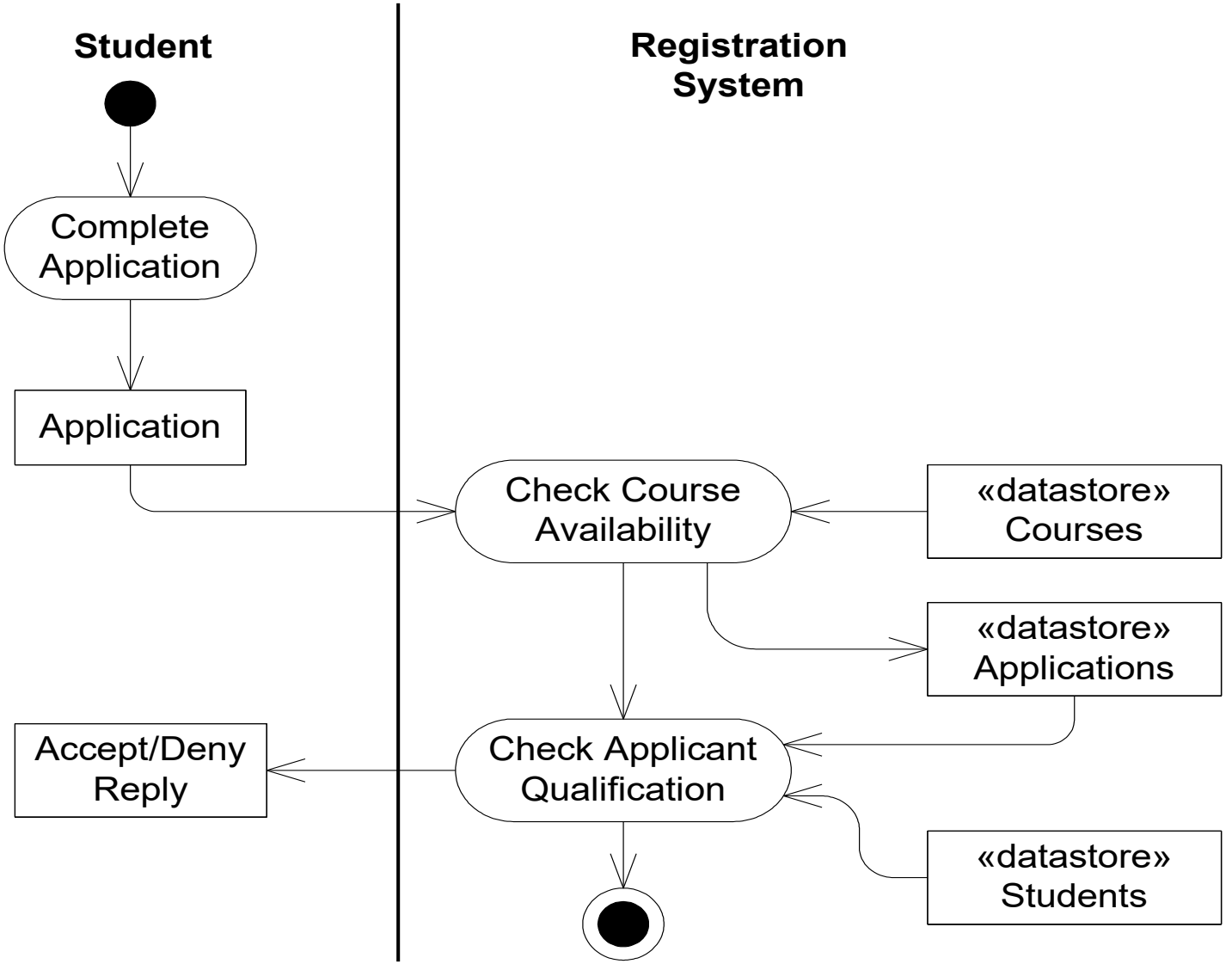
How to apply Activity Diagrams?

- Business Process Modeling
- Data Flow Modeling
- Concurrent programming and Parallel algorithm Modeling

Business process modeling



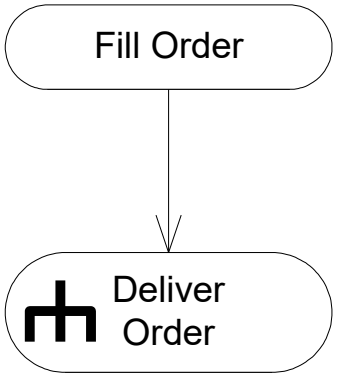
Data flow modeling



Source: "Applying UML and patterns" by Craig Larman

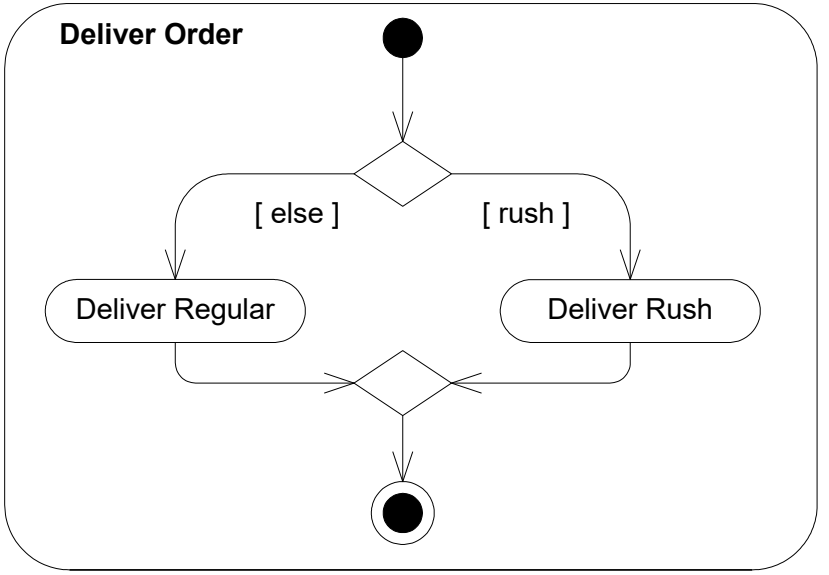
Complex activity diagrams

the “rake” symbol (which represents a hierarchy) indicates this activity is expanded in a sub-activity diagram



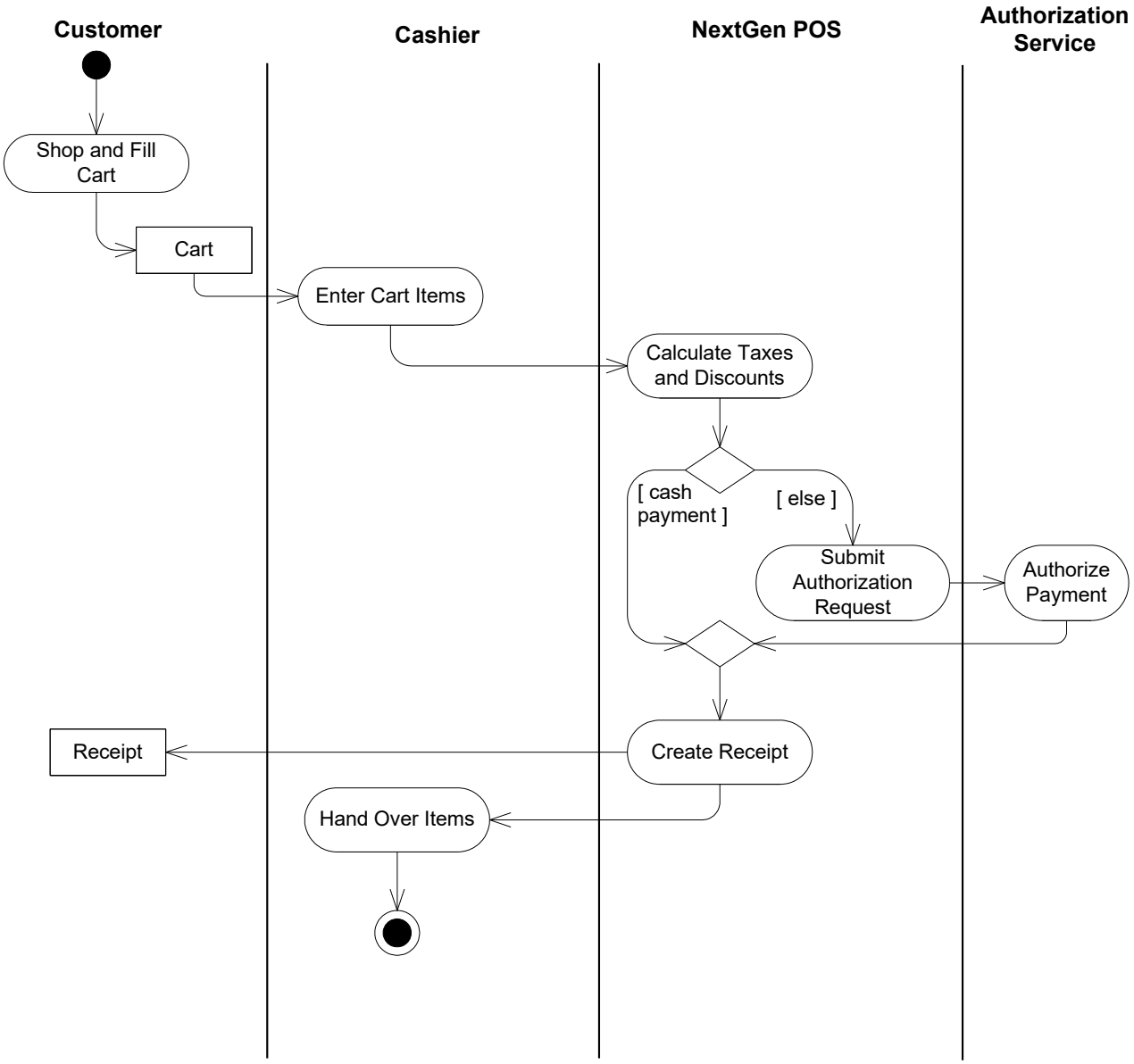
Decision: Any branch happens. Mutual exclusion

Merge: *Any* input leads to continuation. This is in contrast to a *join*, in which case *all* the inputs have to arrive before it continues.



Source: “Applying UML and patterns” by Craig Larman

Process sale activity diagram



Source: "Applying UML and patterns" by Craig Larman

From Workflow to Architecture

Activity Diagrams:

- Visualize complex processes clearly.
- Expose concurrency and dependencies.
- Reveal boundaries between responsibilities

  → candidate components.

Behavioral Element	**Architectural Artifact**
Activity/Action	Component responsibility
Swimlane	Subsystem or component boundary
Control flow	Connector between components

Thank you !